QB-TimeWarp — Change Log

> **Purpose**: Chronological record of all changes made to the project, with reasoning and affected files.

Change 1: Initial Implementation — QBXML SDK Integration

What: Created the foundational project structure with QBXML SDK integration for QuickBooks data migration.

Why: No native tool exists for backward migration from QB 2023 → QB 2021. The QBXML SDK is the only reliable programmatic interface to QuickBooks.

Files Created:
- `QB-TimeWarp.csproj` — Project file targeting .NET 6.0, x86, with NuGet packages (Newtonsoft.Json, Serilog, Spectre.Console)
- `Program.cs` — Main orchestrator with 4-stage pipeline (Export → Transform → Import → Validate)
- `Services/QBConnectionManager.cs` — COM Interop wrapper for QBXML SDK connections
- `Services/DataExporter.cs` — Exports entities from QB 2023 to JSON
- `Services/DataTransformer.cs` — Transforms data for QB 2021 compatibility
- `Services/DataImporter.cs` — Imports transformed JSON into QB 2021
- `Services/DataValidator.cs` — Post-import validation
- `Services/SchemaExtractor.cs` — Extracts field schemas from QB 2021
- `Models/ConfigurationModels.cs` — Configuration POCOs
- `Models/MigrationModels.cs` — Data transfer models
- `Models/ValidationModels.cs` — Validation report models
- `Models/FieldMappingModels.cs` — Field mapping models
- `Models/QBEntityType.cs` — Entity type definitions
- `Helpers/ProgressReporter.cs` — Console progress bars and banners
- `Helpers/TransformFunctions.cs` — Transformation utility functions
- `appsettings.json` — Application configuration
- `Configuration/FieldMappings.json` — Field mapping rules
- `README.md` — Project documentation
- `.gitignore` — Git ignore rules

Configuration: Base `appsettings.json` created with QB 2023/2021 paths, export/import settings, and logging configuration.

Change 2: Inactive Entity Export (`ActiveStatus=All` Filter)

What: Updated `DataExporter.cs` to include inactive entities in the export by adding `ActiveStatus=All` to QBXML queries for entity types that support it.

Why: The client has inactive customers, vendors, items, etc. in QB 2023. Without exporting them, historical transactions referencing these entities would fail to import or lose referential integrity.

Files Modified:
- `Services/DataExporter.cs` — Added `ActiveStatusEntityTypes` HashSet; modified `ExportEntityType()` to inject `<ActiveStatus>All</ActiveStatus>` into QBXML queries; added active/inactive count tracking; added `LogInactiveEntitySummary()` method
- `Models/MigrationModels.cs` — Added `ActiveCount` and `InactiveCount` properties to `ExportedEntitySet`
- `appsettings.json` — Added `"IncludeInactiveRecords": true` to Export section

Configuration Update: Set `Export.IncludeInactiveRecords = true` in `appsettings.json`.

Change 3: Journal Validation for Debit/Credit Matching

What: Added comprehensive journal entry validation to ensure all journal entries balance (total debits = total credits within tolerance).

Why: Accountant's critical requirement. Unbalanced journal entries indicate data corruption and would be rejected by QB 2021. This validation catches issues before they become accounting problems.

Files Modified:
- `Services/DataValidator.cs` — Added journal validation logic that checks each journal entry's debit and credit line totals
- `Models/ValidationModels.cs` — Added journal validation report models
- `Models/ConfigurationModels.cs` — Added `EnableJournalValidation` boolean property (defaults to `true`)
- `appsettings.json` — Added `"EnableJournalValidation": true` to Validation section

Configuration Update: Set `Validation.EnableJournalValidation = true` in `appsettings.json`.

Change 4: Inactive-to-Active Transformation Rule

What: Added transformation rule that converts all inactive entities to active during the Transform stage.

Why: Accountant requirement for clean migration. QB 2021 may reject imports of inactive entities, and the accountant wants all records active for initial review. They can deactivate records manually after verifying the migration.

Files Modified:
- `Services/DataTransformer.cs` — Added logic to set `IsActive=true` on all entities when `ReactivateInactiveEntities` is enabled; tracks reactivation count in `TransformationReport`
- `Models/MigrationModels.cs` — Added `ReactivatedCount` to `TransformationReport`
- `Models/ConfigurationModels.cs` — Added `TransformationRulesConfig` with `ReactivateInactiveEntities` property
- `appsettings.json` — Added `TransformationRules.ReactivateInactiveEntities = true`
- `Program.cs` — Updated to pass `TransformationRules` config to `DataTransformer`; displays reactivation summary

Configuration Update: Added `TransformationRules` section to `appsettings.json`.

Change 5: Class Tracking Preservation

What: Added class tracking preservation — discovers classes used in transactions, checks if they exist in QB 2021, and auto-creates missing ones before import.

Why: Classes are used for cost center reporting (departments, locations, projects). Losing class assignments would break the accountant's P&L by Class reports and other departmental financial reports.

Files Modified:
- `Services/DataTransformer.cs` — Added class discovery logic that scans all transactions for `ClassName` / `ClassRef` fields; collects unique classes
- `Services/DataImporter.cs` — Added `EnsureClassesExist()` method that queries QB 2021 for existing classes, creates missing ones via `ClassAddRq` (handles hierarchical parent:child classes); added `QueryExistingClasses()` and `CreateClassInQB2021()` methods
- `Models/MigrationModels.cs` — Added `ClassTrackingSummary` model
- `Program.cs` — Added Step 4b: Class existence check before transaction import
- `appsettings.json` — Added `TransformationRules.PreserveClassTracking = true`

Configuration Update: Set `TransformationRules.PreserveClassTracking = true` in `appsettings.json`.

Change 6: Accounting Model Matching

What: Added logic to query the accounting method (Cash/Accrual) from QB 2023 and apply it to QB 2021 using `PreferencesModRq`.

Why: If the source uses Accrual basis and the target uses Cash basis, all financial reports in QB 2021 would show incorrect numbers. The accounting method must match for data integrity.

Files Modified:
- `Services/DataImporter.cs` — Added `QueryCompanyPreferences()` to read preferences from QB 2021; added `ApplyAccountingPreferences()` to update accounting method via `PreferencesModRq`
- `Models/MigrationModels.cs` — Added `CompanyPreferences` model with AccountingMethod, ReportBasis, ClassTrackingEnabled, etc.
- `Program.cs` — Added Step 4a: Accounting preferences matching before import; logs source/target comparison
- `appsettings.json` — Added `TransformationRules.MatchAccountingModel = true`

Configuration Update: Set `TransformationRules.MatchAccountingModel = true` in `appsettings.json`.

Change 7: Date/Field Format Preservation

What: Added comprehensive format preservation system for dates, currency, phone numbers, postal codes, and memo fields during transformation. Added corresponding validation.

Why: Data integrity during migration. Dates with timezones cause QB 2021 errors. Currency rounding errors compound across transactions. Postal codes losing leading zeros (01234 → 1234) break mailing. Phone numbers exceeding QB field limits get truncated poorly.

Files Modified:
- `Helpers/TransformFunctions.cs` — Added `FormatPreservationResult` class; added format-specific functions: `PreserveDate()`, `PreserveCurrencyFormat()`, `PreservePhoneFormat()`, `PreservePostalCodeFormat()`, `PreserveMemoFormat()`, `FormatAwareTruncate()`; added detection helpers: `IsDateField()`, `IsCurrencyField()`, `DetectDateFormat()`, `ValidateDateForQB2021()`
- `Services/DataTransformer.cs` — Refactored `ProcessValue()` to attempt format-aware processing first; added `DetectFormatType()` and `ApplyFormatPreservation()` methods; added `LogFormatPreservationSummary()`
- `Services/DataValidator.cs` — Added `ValidateFormatPreservation()` orchestrator; added `ValidateDateFieldFormats()`, `ValidateCurrencyFieldFormats()`, `ValidatePhoneFieldFormats()`, `ValidatePostalCodeFieldFormats()` methods
- `Models/MigrationModels.cs` — Added `FormatPreservationStats` class with per-type counts
- `Models/ValidationModels.cs` — Added `FormatValidationReport` and `FormatValidationIssue` classes
- `Models/FieldMappingModels.cs` — Added `FormatRulesConfig` class; added `FormatType` property to `FieldMapping`; added `FormatRules` to `FieldMappingsConfig`
- `Configuration/FieldMappings.json` — Added `formatRules` section with preservation toggles and field patterns
- `README.md` — Added comprehensive “Field Format Preservation” documentation section

Configuration Update: Added `formatRules` section to `Configuration/FieldMappings.json` with all format preservation toggles enabled by default.

Change 8: QB 2023 Path Correction in Configuration

What: Updated `appsettings.json` to use the correct QB 2023 executable path.

Why: The original path pointed to the install directory; the corrected path points to the actual executable `QBWPremierAccountant.exe` which is needed for SDK connection.

Files Modified:
- `appsettings.json` — Changed `QB2023.InstallPath` from `C:\Program Files\Intuit\QuickBooks 2023` to `C:\Program Files\Intuit\Quickbooks 2023\QBWPremierAccountant.exe`

Change 9: Comprehensive Project Documentation

What: Created a full documentation suite for project context retention across sessions.

Why: AI sessions are ephemeral. Without persistent documentation, each new session loses context about requirements, design decisions, connection details, and implementation history.

Files Created:
- `PROJECT_CONTEXT.md` — Complete project overview, requirements, design decisions, transformation rules
- `SESSION_RESUME.md` — Connection details, credentials, paths, build instructions
- `CHANGE_LOG.md` — This file: chronological change history
- `ARCHITECTURE.md` — System architecture, service layers, data flow
- `TESTING_CHECKLIST.md` — Comprehensive testing steps and expected results
- `README_FOR_NEW_SESSIONS.md` — Quick start guide for resuming work

Change 10: Working Copy System — Original File Protection

What: Added automatic working copy creation system that copies original QB company files from Desktop to dedicated Working directories before any operations begin. Multiple safety layers prevent any modification to original files.

Why: Original QuickBooks company files (Joshua's Gold Coast, Blank Template, Air Masters) must NEVER be directly modified. The working copy system ensures all operations use disposable copies while originals remain pristine on the Desktop.

Files Modified:
- `Services/WorkingDirectoryManager.cs` — NEW: Manages working directory creation, file copying, verification, and cleanup
- `Models/ConfigurationModels.cs` — Added `SourceFilesConfig`, `TargetFilesConfig`, `WorkingDirectoriesConfig` classes
- `appsettings.json` — Added `SourceFiles`, `TargetFiles`, `WorkingDirectories` sections; updated `CompanyFilePath` to Working paths
- `Program.cs` — Added working copy initialization before QB operations; added `--cleanup` and `--refresh` commands; added safety exception handling
- `Services/QBConnectionManager.cs` — Added `ValidateCompanyFilePath()` with protected Desktop folder patterns; blocks connections to originals
- `SAFETY_FEATURES.md` — NEW: Comprehensive documentation of all safety layers
- `SESSION_RESUME.md` — Added working copy folder structure section
- `TESTING_CHECKLIST.md` — Added Stage 0: Working Copy Verification
- `README.md` — Added Working Copy System section and safety feature description
- `CHANGE_LOG.md` — This entry

Configuration: Ensure `SourceFiles.DesktopFolder` and `TargetFiles.DesktopFolder` point to the correct Desktop folders. `WorkingDirectories.AutoCreateWorkingCopies` should be `true`.

New CLI Options: `--refresh` (force re-copy), `--cleanup` (delete working dirs)

New Exit Code: 99 = Safety violation (attempted operation on original file)
